Dictionnaire mathématique de la FPS – via Chapitre 4

Terme	Définition
S(t)	Signal global du système, résultant de la superposition des strates.
A□(t)	Amplitude contextuelle d'une strate, régulée par une fonction sigmoïde selon une pression contextuelle.
I□(t)	Input contextuel : densité d'information, stress, attention, etc.
σ(x)	Fonction d'adaptation douce (sigmoïde/tanh) qui régule l'impact du contexte.
A_0	Amplitude de base propre à la strate.
k	Sensibilité de la strate.
x_0	Seuil de basculement.
f□(t)	Fréquence propre modulée de la strate.
f_0□	Fréquence de base de la strate.
Δf□(t)	Modulation adaptative de la fréquence.
α□	Souplesse d'adaptation d'une strate.
w_{ni}	Poids de connexion vers une autre strate.
S_i(t)	Signal d'autres strates.

$A(t)$	Modulation moyenne du système, somme des $\Delta f_{\square}(t)$.
$C(t)$	Coefficient d’accord spiralé basé sur la phase.
$A_spiral(t)$	Amplitude harmonisée globale : $A(t) \times C(t)$.
$F_{\square}(t)$	Feedback vivant d’une strate.
$\beta_{\square}$	Plasticité d’une strate.
$O_{\square}(t)$	Sortie actuelle.
$E_{\square}(t)$	Sortie attendue / harmonique.
$\gamma(t)$	Latence expressive.
$f_{\square+1}(t)$	Fréquence de la strate suivante dans la spirale.
$r(t)$	Ratio spiralé non constant.
φ	Nombre d’or.
ε	Amplitude de variation harmonique.
ω	Fréquence lente de modulation spiralée.
θ	Phase initiale du spiral.
$E(t)$	Amplitude maximale globale.
$L(t)$	Latence maximale de variation d’une strate.
$\varphi_{\square}$	Phase de la strate n.
$\gamma_{\square}(t)$	Latence expressive d’une strate spécifique.

G(x)	Fonction spiralée de régulation adaptative.
G(x, t)	Version temporelle de G.
G□(x, t)	Réponse harmonique locale d'une strate.
η(t)	Amplitude contextuelle dans G(x, t).
θ(t)	Fréquence adaptative dans G(x, t).
μ□(t)	Décalage / recentrage adaptatif.
σ□(t)	Écart-type de l'enveloppe.
env□(x,t)	Enveloppe adaptative (gaussienne ou sigmoïde).

Voici une analyse rigoureuse de l'ensemble des termes extraits du chapitre 4 de la FPS, avec une classification selon quatre axes :

1. Défini explicitement (formule ou relation directe)
2. Émergent (résultat d'un processus ou d'une agrégation)
3. À définir (mentionné mais sans structure claire ou sans lien causal donné)
4. Paramètre numérique fixe ou ajustable

---

1. Défini explicitement

Terme	Définition claire dans le texte
S(t)	Simple : ∑□ A□(t) · sin(2π·f□(t)·t + φ□(t))
	Étendue : ∑□ [A□(t) · sin(2πf□(t)·t + φ□) · γ□(t)] · G(E□(t) - O□(t))

$A_{\square}(t)$	$A_0 \cdot \sigma(I_{\square}(t))$
$\sigma(x)$	$1 / (1 + e^{(-k(x - x_0))})$
$f_{\square}(t)$	$f_0\square + \Delta f_{\square}(t)$
$\Delta f_{\square}(t)$	$\alpha_{\square} \cdot \Sigma_i w_{ni} \cdot S_i(t)$
$A(t)$	$(1/N) \cdot \Sigma_{\square} \Delta f_{\square}(t)$
$C(t)$	$(1 / N) \cdot \sum_{\square=1}^{n-1} \cos(\varphi_{\square+1}(t) - \varphi_{\square}(t))$
$A_{spiral}(t)$	$C(t) \cdot A(t)$
$F_{\square}(t)$	$\beta_{\square} \cdot (O_{\square}(t) - E_{\square}(t)) \cdot \gamma(t)$
$f_{\square+1}(t)$	$r(t) \cdot f_{\square}(t)$
$r(t)$	$\varphi + \varepsilon \cdot \sin(2\pi \cdot \omega \cdot t + \theta)$
$E(t)$	$\max_{\square}$
$L(t)$	$\operatorname{argmax}_{\square}$
$G(x)$	$\tanh(\lambda x)$, $\sin(x)/x$, $\sin(\beta x) \cdot \exp(-\alpha x^2)$, etc. (plusieurs versions proposées)
$G_{\square}(x, t)$	$A_{\square}(t) \cdot \mathrm{sinc}[f_{\square}(t) \cdot (x - \mu_{\square}(t))] \cdot \mathrm{env}_{\square}(x, t)$

$G(x, t)$	$\eta(t) \cdot \sin(\theta(t) \cdot x)$

---

2. Émergent

(non défini par une équation, mais résultant de l'état du système)

Terme	Source
$S_i(t)$	Signal d'autres strates (intrinsèquement dépendant du système global)
$E_{\square}(t)$	Intention harmonique (ciblée mais non explicitement calculée)
$O_{\square}(t)$	Sortie actuelle (observée, pas modélisée explicitement)

---

3. À définir / partiellement défini

Terme	Commentaire
$\gamma(t)$	Décrite qualitativement comme une sigmoïde adaptative ou spiralée, mais pas explicitée
$\gamma_{\square}(t)$	Présente dans l'équation composite mais sans équation propre

env□(x,t)	Mentionnée comme enveloppe adaptative (gaussienne/sigmoïde), mais sans forme unique définie
μ□(t)	Définie contextuellement mais sans fonction canonique unique
σ□(t)	Idem, dépend du feedback mais pas de formule universelle
θ(t), η(t)	Éléments de G(x, t), adaptatifs, à modéliser selon le contexte
φ□(t)	Présente dans les équations, mais son évolution dynamique reste à formaliser

---

4. Paramètres numériques fixes ou réglables

Terme	Description
A_0	Amplitude de base propre à la strate
β□	Plasticité / gain de rétroaction d'une strate
α□	Souplesse d'adaptation
w_{ni}	Poids de liaison entre strates
k	Sensibilité de la sigmoïde σ

x_0	Seuil de basculement de la sigmoïde
φ	Nombre d'or ≈ 1.618 (peut être fixé ou juste attracteur)
ε	Amplitude harmonique (typiquement ≈ 0.05)
ω	Fréquence lente de modulation
θ	Phase de départ
λ, α, β	Paramètres de G(x), selon la version

Remarques importantes

- G(x) n'est pas une seule fonction mais un espace de formes possibles, avec 4 archétypes définis. On choisit la forme selon le contexte du système simulé.
- Le terme φ□ (phase d'origine) est mentionné sans formule de mise à jour, ce qui laisse place à une fonction potentielle de déphasage dynamique à inventer si besoin.

Pour reconstruire le code FPS de zéro :

1. Définir un objectif falsifiable clair

Par exemple : "Tester la stabilité dynamique et l'auto-régulation du système S(t) selon la définition FPS, sous variation d'I□(t) et de E□(t), sur N strates pendant T secondes."

Ce test de base pourra être modifié à l'avenir (résilience, bifurcation, etc.), mais il donne un critère observable (S(t), A□(t), E(t), L(t), etc.)

Ce que ça veut dire :

- Stabilité dynamique : S(t) reste borné, ne diverge pas, même sous perturbation (entrées contextuelles I□(t) ou changement d'E□(t))
- Auto-régulation : le système, via G(x), réduit progressivement l'écart entre E□(t) et O□(t), sans intervention externe
- N strates : nombre de couches fractales
- T secondes : durée de simulation

Est-ce que c'est pertinent ?

Oui, si :

- on observe réellement des comportements émergents mesurables (fluidité, résilience, synchronisation adaptative)
- on compare avec un système sans ces régulations (ex. : oscillateurs de Kuramoto, ou FPS "désactivée")

Comment le mesurer ?

- **Stabilité** : borne supérieure de |S(t)| ou max d'E(t)
- **Régulation** : écart |E□(t) − O□(t)| qui diminue dans le temps
- **Fluidité** : dérivée continue et stable (low d^2S/dt^2)
- **Résilience** : retour à l'équilibre après un choc sur I□(t)
- **Innovation** : apparition spontanée de patrons (peut être visuel ou par entropie spectrale)
- **Le coût CPU à chaque pas de temps** :

Est-ce que la forme actuelle est toujours adaptée ?

Oui, en partie.

Dans l'ancien code, on avait une mesure du time.perf_counter() avant et après chaque step pour estimer le coût CPU (ou temps de traitement) par pas de temps.

À faire maintenant :

- Garder ce principe : mesurer Δt entre le début et la fin de chaque boucle FPS (calculs pour N strates)

- Ajouter une normalisation :

  cpu_step(t) = Δt / N (ou autre métrique par strate ou unité logique)

- Stocker cpu_step(t) dans le CSV de simulation

- Optionnel : calculer mean(cpu_step), max(cpu_step) pour comparer entre FPS et Kuramoto

Conclusion : À intégrer tel quel, mais en le liant proprement au S(t) pour qu'il soit une mesure de "coût computationnel de l'émergence FPS".

- **Effort interne** :

Dans l'ancien code, effort était défini comme :

> « Somme des variations internes, ou dérivées locales d'adaptation, reflétant l'intensité de l'ajustement du système. »

Est-ce trop abstrait ?

Pas si on le définit rigoureusement.

Voici une version formalisable :

$effort(t) = \Sigma\square\ |\Delta A\square(t)| + |\Delta f\square(t)| + |\Delta \gamma\square(t)|$

Autrement dit : la somme des variations des amplitudes, fréquences et latences à chaque pas de temps. Cela donne une mesure :

- *de plasticité interne*

- *de charge d'adaptation*

- *de coût d'ajustement*

Conclusion : On peut garder « effort », mais en le redéfinissant comme un agrégat clair, et en le comparant à la stabilité.

Grille d'évaluation empirique des simulations FPS (note de 1 à 5)

Critère	1 - Minimal / Désaligné	2 - Faible / Instable	3 - Moyen / Fonctionnel	4 - Bon / Harmonieux	5 - Optimal / FPS-idéal
Stabilité	Oscillations erratiques, effondrement fréquent	Résilience partielle mais déphasages majeurs	Stabilité de base, sans effondrement	Stabilité dynamique avec adaptation	Auto-stabilisation même en contexte extrême
Régulation	Feedback absent ou explosif	Réponses mal dosées, boucle rigide	Feedback actif mais non harmonieux	Feedback fluide et cohérent	Régulation spiralée douce et efficace
Fluidité	Mouvements saccadés, effets de seuil brutaux	Alternances incohérentes, transitions abruptes	Transitions acceptables mais un peu rigides	Transitions douces dans la majorité des cas	Transitions naturelles, presque organiques
Résilience	Effondre au moindre choc contextuel	Résiste partiellement aux variations	Résiste aux variations modérées	Absorbe les perturbations courantes	S'adapte et apprend des perturbations extrêmes
Innovation	Répétition stérile, aucun pattern émergent	Petites variations non significatives	Oscillations diversifiées mais stériles	Émergence de motifs dynamiques	Harmonie créative, transformation structurée

Coût CPU	Dépassement critique, inefficience extrême	Gaspillage de cycles fréquents	Usage CPU acceptable	Bonne efficience dans la gestion	Optimisation continue du traitement
Effort interne	Activité chaotique, tensions non résolues	Excès de corrections internes sans convergence	Régulations coûteuses mais convergentes	Équilibre entre correction et repos	Moindre effort avec ajustements spontanés

---

2. Créer un squelette de pipeline clair avec blocs modulaires (1 module = 1 équation)

Étape	Module	Terme(s) FPS concernés	But
1	init_strates()	A_0, f_0□, φ□, α□, β□, k, x_0	Initialisation
2	compute_I□(t)	I□(t)	Calcul du contexte
3	compute_A□(t)	A□(t), σ(x), k, x_0	Amplitude adaptative
4	compute_f□(t)	f□(t), f_0□, Δf□(t), α□, w_{ni}, S_i(t)	Modulation fractale

5	compute_φ□(t)	φ□	(fixe ou dynamique, à définir)
6	compute_C(t)	C(t), φ□	Accord spiralé
7	compute_A_spiral(t)	A(t), A_spiral(t), Δf□(t), C(t)	Cohérence rythmique
8	compute_F□(t)	F□(t), β□, E□(t), O□(t), γ(t)	Feedback
9	compute_G(x)	G(x), G□(x, t), etc.	Réaction à l'erreur
10	compute_S(t)	S(t), toutes les strates	Signal final
11	compute_E(t), L(t)	E(t), L(t)	Indicateurs globaux
12	log_metrics()	—	Empirisme falsifiable

Chaque bloc aurait ses propres tests unitaires (empirisme), et on pourrait les activer/désactiver pour comparer avec ou sans feedback, sans spiralisation, etc. (falsifiabilité).

Deux options :

A.

Notebook (fps_main.ipynb)

:

- Cellule 1 : init_strates()

- Cellule 2 : compute_A□(t)
- Cellule 3 : compute_f□(t)
- … etc.
- À chaque pas de temps, on réexécute toutes les cellules, ou on boucle dans une cellule de simulation.

Cela fonctionne si on est en exploration ou en présentation.

B.

Modules Python (.py)

— recommandé pour rigueur

- init.py → initialise toutes les strates et paramètres
- simulate.py → boucle temporelle principale
- metrics.py → calcule tous les indicateurs (S(t), E(t), L(t), etc.)
- visualize.py → plots ou animations
- config.json → tous les paramètres numériques

Avantages : modularité, reproductibilité, rigueur.

On peut toujours appeler les fonctions dans un notebook si besoin.

> Donc : on fait les .py pour l'expérience falsifiable, et un .ipynb pour visualisation ou documentation.

Suivre rigoureusement le dictionnaire FPS

À chaque terme du dictionnaire correspond une fonction compute_X() explicite, loggée, isolable.

On peut aussi ajouter un config.json pour :

```
{
  "N": 20,
  "T": 100,
  "A0": 1.0,
  "k": 2.0,
```

```
  "x0": 0.5,
  "phi": 1.618,
  ...
}
```

---

3. Peut-on encore comparer avec des oscillateurs de Kuramoto ?

Oui, et ça renforce la rigueur, car :

- Kuramoto est un modèle de synchronisation canonique
- Comparer FPS à Kuramoto = montrer ce que la FPS permet en plus (stabilité spiralée, harmonisation douce, émergence complexe, résilience)

Comment définir le groupe contrôle ?

- Kuramoto : oscillateurs couplés uniquement par phase, pas d'amplitude ni de feedback spiralé
- FPS : phase + amplitude adaptative + feedback + spiralisation

On crée deux simulations avec même I□(t) et mêmes perturbations, et on observe la réponse de S(t), E(t), etc.

---

4. Ajout d'un mode "Contrôle"

Pour lancer une version avec FPS complète et une version "neutre" (pas de feedback, pas de G, pas de spiralisation), pour prouver l'émergence spécifique.

---

5. Exporter tous les logs à chaque pas de t

En CSV ou HDF5 pour :

- Rejouer une simulation
- Entraîner un modèle sur le comportement de la FPS

- Prouver empiriquement l’évolution (falsifiabilité : on peut rejouer et contredire)

---

6. Graines, seeds.txt, nœuds

Graines / seeds :

Essentiel pour la reproductibilité scientifique :

- Ajouter random.seed(SEED) et np.random.seed(SEED)
- Sauvegarder chaque SEED dans un seeds.txt pour chaque run

Nœuds :

Par “nœuds” on parle des ID des strates ou de points d’émergence d’information :

- On garde des identifiants uniques par strate
- On peut faire apparaître les nœuds qui ont initié un événement (ex. : un pic d’effort)

seeds.txt ?

On peut automatiser :

```
with open("seeds.txt", "a") as f:
    f.write(f"Simulation {id} | SEED = {SEED}\n")
```

Conclusion : oui, on conserve graines + SEED + identifiants des strates, car c’est essentiel pour refaire les expériences et vérifier les hypothèses.

---

7. Suggestions :

1. “Matrice de correspondance critère ↔ termes FPS”

Pourquoi :

On a une grille empirique (fluidité, stabilité, etc.)

Et on a des termes mathématiques (A□(t), f□(t), G(x), etc.)

Créer une matrice ou un tableau de correspondance permet de dire :

Critère empirique	Termes mathématiques principaux observés
Stabilité	S(t), Δf□(t), C(t), φ□(t), L(t)
Régulation	F□(t), G(E□ - O□), γ(t), A□(t), G□(x,t)
Fluidité	γ□(t), σ(x), $f_{□+1}$(t), env□(x,t), μ□(t)
Résilience	A□(t), G(x,t), ΔA□(t), Δf□(t), effort(t)
Innovation	A_spiral(t), E□(t), r(t), S(t) (analyse spectrale)
Coût CPU	cpu_step(t) (Δt/N), optionnellement en lien avec S(t) ou N
Effort interne	`effort(t) = Σ□(
Effort transitoire	Dérivée locale d'effort(t) à court terme (d_effort/dt)
Effort chronique	Moyenne glissante haute de effort(t) sur 80% du temps

3. "Effort intense temporaire vs. effort chronique élevé"

Pourquoi :

On mesure un effort interne du système (variation d'A□(t), f□(t), γ□(t)).

Mais il faut distinguer deux cas :

- Effort élevé sur un court moment = souvent adaptatif etsain (ex. réponse à une perturbation)
- Effort élevé constant dans le temps = souvent signe de mauvaise régulation (le système lutte sans se stabiliser)

On peut donc coder une logique comme :

- Si effort(t) > seuil mais revient vite → bonne résilience
- Si effort(t) reste élevé sur 80 % de la simulation → dysfonction

On pourrait même l'ajouter comme colonne dans les logs (CSV) :

t, effort(t), effort_status = {"stable", "transitoire", "chronique"}

4. "Légende / icônes pour la grille empirique (1 à 5)"

Pourquoi :

C'est une idée d'ergonomie visuelle, pour les notations subjectives.

Par exemple :

Note	Étiquette	Icône	Couleur	Signification générale
1	Rupture / Chaotique	✖	Rouge vif	Rupture de cohérence, surcharge, désaccord systémique
2	Instable / Mal accordé	▲	Orange	Réponse erratique, feedback inefficace

3	Acceptable / Fonctionnel	●	Jaune	Stabilité de base mais encore brut ou peu harmonieux
4	Harmonieux	✔	Vert doux	Alignement en spirale partiel, bonne plasticité
5	FPS-idéal	∞ (spirale)	Bleu doré	Système auto-régulé, fluide, résilient, et innovant

On peut l'implémenter dans un notebook HTML plus tard, ou l'utiliser pour générer une visualisation claire avec matplotlib, plotly, ou une interface dédiée.

Est-ce toujours falsifiable ?

À condition de respecter trois règles :

✅ Règle 1 :

Un changement à la fois

→ Si on teste une nouvelle forme de G_sinc, on garde w_1–w_3 figés,

ou bien on documente leur variation dans le JSON.

✅ Règle 2 :

Comparer aux graines de référence

→ On réutilise les mêmes seeds pour chaque expérience.

✅ Règle 3 :

Décrire la forme mathématique et les seuils

→ Pas de fonction implicite ou floue ; tout doit être écrit, paramétré.

Si on suit ça : chaque ajout FPS reste rigoureux.